GERMAN UNIVERSITY IN CAIRO

Faculty of Engineering & Materials Science
Mechatronics Engineering Department

MCTR911 – ROBOTICS PROGRAMMING

MILESTONE 03

Forward and Inverse Velocity and Acceleration Kinematics of a UR5e Robot

Python Implementation, MuJoCo Validation, and Smooth Motion Demonstration

Application: Simulated Autonomous Surgical Tool Handover

Team member	Student ID
1. Abdelrahman Emad	58-2133
2. Basmala Yasser	58-2375
3. Omar Khattab	58-24063
4. Samer Ramzy	58-19523
5. Hebatalla Mohamed	58-2372
6. Mariam Ayman	58-13210

Abstract

This report documents the third milestone of the MCTR911 Robotics Programming project using a six-degree-of-freedom UR5e manipulator in a MuJoCo surgical-workstation scene. Building on the standard Denavit–Hartenberg (DH) model and forward position kinematics established in Milestone 02, the work derives and implements the robot's forward and inverse velocity kinematics and forward and inverse acceleration kinematics in Python. A world-frame 6 × 6 geometric Jacobian is constructed analytically from the DH joint axes and origins. The inverse velocity and acceleration problems are solved using a damped least-squares pseudoinverse based on singular-value decomposition. The analytical Jacobian time derivative is also implemented and checked against a central finite-difference approximation.

The implementations are validated against MuJoCo at five prescribed joint configurations. All five velocity tests and all five acceleration tests pass their configured tolerances. The previously established forward-position tests and three Jacobian-transpose inverse-position tests are also summarized to show continuity across the position, velocity, and acceleration levels. A separate smooth point-to-point simulation uses a quintic joint-space reference and the existing MuJoCo position actuators. The supplied 2 ms dataset contains 4,001 samples over a 6 s movement and 2 s final hold. The measured joint-speed norm rises from near zero to approximately 0.144 rad/s around the midpoint and returns to near zero at the end. Python and MuJoCo kinematic quantities calculated at the same actual state agree to errors far below the configured tolerances.

The report distinguishes kinematic-equation agreement from actuator tracking. In particular, the instantaneous MuJoCo generalized-acceleration snapshot and the finite-difference derivative of sampled joint velocity do not coincide in the smooth-motion experiment. The finite-difference acceleration follows the reference closely, while the instantaneous qacc comparison remains a separate diagnostic. This distinction is retained in the results rather than treating the two acceleration definitions as interchangeable.

Keywords: UR5e, geometric Jacobian, velocity kinematics, acceleration kinematics, damped least squares, MuJoCo, quintic trajectory, validation.

Report organization

The report first defines the robot model and coordinate conventions, then presents the mathematical derivation and Python implementation. The validation procedure and numerical results follow. The smooth-motion experiment is discussed separately because agreement between kinematic equations and agreement between an actuator-driven trajectory and its reference are different questions. The final sections summarize limitations, reproducibility, and submission evidence.

Contents

1. Milestone Objectives and Scope
2. Robot Model and Coordinate Conventions
3. Kinematic Preliminaries
4. Forward and Inverse Velocity Kinematics
5. Forward and Inverse Acceleration Kinematics
6. Python Implementation and Validation Procedure
7. MuJoCo Velocity and Acceleration Validation Results
8. Smooth Point-to-Point Motion Demonstration
9. Dynamic Kinematic Agreement and Acceleration Measurement
10. Surgical Scene, GUI, and Reproducibility
11. Discussion and Limitations
12. Conclusion

References

Submission Completion Checklist

1. Milestone Objectives and Scope

The MCTR911 Milestone 03 brief requires the team to derive forward and inverse velocity kinematics and forward and inverse acceleration kinematics in Python; validate the kinematic equations against the selected simulator by prescribing joint states and comparing end-effector position, velocity, and acceleration; and continue developing the application GUI/environment. The required submission includes a report, a narrated simulation-validation video, code, GUI videos, and videos testing the kinematic equations on the model [1].

The selected application remains a simulated autonomous surgical-tool handover workstation. The robot is a six-revolute-joint UR5e. Milestone 03 extends the position-level kinematics developed in Milestone 02; it does not replace the robot model or introduce a new manipulator.

Course requirement	Evidence documented in this report
Derive forward and inverse velocity kinematics in Python	Sections 4.2–4.3 and the five-configuration MuJoCo validation in Section 6.
Derive forward and inverse acceleration kinematics in Python	Sections 4.4–4.6 and the five-configuration MuJoCo validation in Section 6.
Validate kinematic equations against the simulator	Position results carried forward from Milestone 02, plus velocity and acceleration tests at five configurations.
Continue the GUI/environment development	The updated surgical MuJoCo scene is used for the smooth-motion simulation; GUI demonstration videos remain a separate submission item.
Submit report, code, and videos	Reproducibility commands and file organization are listed in Section 10.

1.1 Scope and interpretation

The main technical contribution of this milestone is a consistent kinematic pipeline from joint states to end-effector motion and back. The validation scripts load the MuJoCo scene directly through the MuJoCo Python API and compare quantities expressed at the named attachment_site. The smooth-motion script also steps the physics model and records actual joint states. These tests are simulation-based; they do not constitute measurements from a physical UR5e robot.

The smooth-motion demonstration uses the existing position actuators. It does not send direct joint-velocity or joint-acceleration commands to velocity or acceleration actuators. The distinction is important when describing the video and interpreting the acceleration plots.

2. Robot Model and Coordinate Conventions

2.1 UR5e model and joint order

The selected robot is a six-degree-of-freedom UR5e manipulator with six revolute joints. The joint order used by the Python kinematics implementation is:

- shoulder_pan_joint
- shoulder_lift_joint
- elbow_joint
- wrist_1_joint
- wrist_2_joint
- wrist_3_joint

The end-effector reference is the MuJoCo site named attachment_site. The selected DH base frame coincides with the robot-base/world reference used by the MuJoCo scene. Consequently, the Python position, linear velocity, angular velocity, linear acceleration, and angular acceleration are compared in the same world coordinate frame as the simulator site quantities.

Angles are in radians, joint rates in rad/s, joint accelerations in rad/s^2, positions in metres, linear velocities in m/s, and linear accelerations in m/s^2. The six-dimensional twist is ordered [vx, vy, vz, ωx, ωy, ωz]^T. Its time derivative is ordered [ax, ay, az, αx, αy, αz]^T.

2.2 Standard DH parameters carried forward from Milestone 02

The velocity and acceleration implementations reuse the team's existing standard-DH forward-kinematics implementation. The following table preserves the frame assignment and signed link dimensions documented in the Milestone 02 report [2].

Joint i	θ_i	d_i (m)	a_i (m)	α_i (rad)
1	q_1	0.163	0	+π/2
2	q_2	0	−0.425	0
3	q_3	0	−0.392	0
4	q_4	0.134	0	+π/2
5	q_5	0.100	0	−π/2
6	q_6	0.100	0	0

These values belong to the team's selected frame convention. In particular, the negative a_2 and a_3 values are intentional signed distances. They should not be combined with a different UR5e DH convention unless the frame definitions and joint offsets are transformed consistently.

3. Kinematic Preliminaries

3.1 Forward position kinematics

Let $q = [q_1, q_2, \ldots, q_6]^T$ denote the joint-angle vector. The homogeneous transform from the base frame to the attachment site is obtained by multiplying the six standard-DH transforms:

$$^0T_6(q) = {}^0T_1(q_1)\ {}^1T_2(q_2)\ {}^2T_3(q_3)\ {}^3T_4(q_4)\ {}^4T_5(q_5)\ {}^5T_6(q_6) \quad (1)$$

The translation part gives the attachment-site position p(q), while the upper-left 3 × 3 block gives its orientation. This is the position-level foundation used to construct and validate the Jacobian.

3.2 Geometric Jacobian

For a revolute joint i, let o_{i-1} be the origin of the corresponding preceding DH frame and z_{i-1} its z-axis, both expressed in world coordinates. If p is the attachment-site position, the linear and angular parts of the i-th Jacobian column are:

$$J_v[:, i] = z_{i-1} \times (p - o_{i-1}) \qquad J\omega[:, i] = z_{i-1} \quad (2)$$

Stacking the six columns produces the 6 × 6 geometric Jacobian $J(q) = [J_v^T, J\omega^T]^T$. The implementation calculates each joint origin and axis while traversing the DH transform chain. A separate central finite-difference Jacobian helper is included as an independent numerical check.

4. Forward and Inverse Velocity Kinematics

4.1 Forward velocity kinematics

The forward velocity problem maps the six joint rates to the end-effector linear and angular velocity. With the world-frame twist convention defined above:

$$\boldsymbol{\nu} = [\mathbf{v}^{\mathbf{T}} \;\; \boldsymbol{\omega}^{\mathbf{T}}]^{\mathbf{T}} = \mathbf{J}(\mathbf{q})\,\dot{\mathbf{q}} \quad (3)$$

The first three components of $\boldsymbol{\nu}$ are the translational velocity of the attachment_site origin. The last three components are its angular velocity expressed in world axes. In the Python implementation, this operation is a matrix-vector product between the geometric Jacobian and the joint-velocity vector.

4.2 Inverse velocity kinematics

The inverse velocity problem starts with a desired end-effector twist $\boldsymbol{\nu}_d$ and finds a joint-rate vector $\dot{q}$ that reproduces it. Rather than directly inverting J, the implementation uses a damped least-squares (DLS) pseudoinverse. For the singular-value decomposition $J = U\Sigma V^T$, the damped inverse is:

$$\mathbf{J}\dagger\lambda = \mathbf{V}\,\mathrm{diag}(\,\sigma_i / (\sigma_i^2 + \lambda^2)\,)\,\mathbf{U}^{\mathbf{T}} \quad (4)$$

$$\dot{\mathbf{q}} = \mathbf{J}\dagger\lambda\,\boldsymbol{\nu}_d \quad (5)$$

The default damping parameter is $\lambda = 10^{-5}$. If damping is disabled, singular values below the configured cutoff of 10^{-10} are discarded. Damping limits the amplification of joint rates near singular configurations, but it can introduce a small residual between the requested and reconstructed twist. Therefore, the reconstructed end-effector velocity is checked after solving the inverse problem.

This inverse calculation is a kinematic least-squares solution. It does not enforce joint-speed limits, collision constraints, or actuator limits, and it is not a motor controller.

5. Forward and Inverse Acceleration Kinematics

5.1 Time derivative of the Jacobian

Acceleration kinematics requires the time derivative of the geometric Jacobian. For column i, the geometric Jacobian is built from the joint axis z_{i-1} and origin o_{i-1}. Differentiating the linear and angular parts gives:

$$\mathbf{J}_v[:, i] = \dot{\mathbf{z}}_{\mathbf{i-1}} \times (\mathbf{p} - \mathbf{o}_{\mathbf{i-1}}) + \mathbf{z}_{\mathbf{i-1}} \times (\dot{\mathbf{p}} - \dot{\mathbf{o}}_{\mathbf{i-1}}) \quad (6)$$

$$\mathbf{J}\omega[:, i] = \dot{\mathbf{z}}_{\mathbf{i-1}} \quad (7)$$

The angular rate of joint axis i caused by the preceding joints is $\omega_{before,i} = \Sigma_{k(k<i)}\, z_{k-1}\, \dot{q}_k$. Therefore, $\dot{z}_{i-1} = \omega_{before,i} \times z_{i-1}$. The corresponding joint-origin velocity is calculated as $\dot{o}_{i-1} = \Sigma_{k(k<i)}\, [z_{k-1} \times (o_{i-1} - o_{k-1})]\, \dot{q}_k$. The attachment-site linear velocity $\dot{p}$ is obtained from $J_v\,\dot{q}$.

The analytical $\dot{J}$ implementation is checked independently using a central finite difference along the instantaneous joint-velocity direction:

$$\mathbf{J}(\mathbf{q}, \dot{\mathbf{q}}) \approx [\mathbf{J}(\mathbf{q} + \mathbf{h}\dot{\mathbf{q}}) - \mathbf{J}(\mathbf{q} - \mathbf{h}\dot{\mathbf{q}})] \,/\, (2\mathbf{h}) \quad (8)$$

The finite-difference expression is used as a validation check; the analytical derivative is used by the forward and inverse acceleration functions.

5.2 Forward acceleration kinematics

Differentiating the forward velocity relationship $\mathcal{V} = J(q)\dot{q}$ gives the six-dimensional twist derivative:

$$\dot{\mathcal{V}} = J(q)\,\ddot{q} + \dot{J}(q, \dot{q})\,\dot{q} \quad (9)$$

The vector $\dot{\mathcal{V}}$ contains the world-frame linear acceleration of the attachment-site origin and the world-frame angular acceleration. Its angular part is the derivative of the world-frame angular-velocity vector; it is not an Euler-angle second-derivative vector.

5.3 Inverse acceleration kinematics

Given a desired end-effector twist derivative $\dot{\mathcal{V}}_d$, the inverse acceleration problem first removes the contribution caused by the changing Jacobian, then solves for joint accelerations:

$$\ddot{q} = J^{\dagger\lambda}\,[\dot{\mathcal{V}}_d - \dot{J}(q, \dot{q})\,\dot{q}] \quad (10)$$

The same DLS/SVD strategy is used as in inverse velocity kinematics. The acceleration solution is a least-squares kinematic solution. It does not calculate actuator torques and does not impose joint-acceleration, collision, or actuator limits.

6. Python Implementation and Validation Procedure

6.1 Implementation modules

File	Responsibility
Milestone 03/kinematics/ur5e_jacobian.py	Constructs the full 6 × 6 world-frame geometric Jacobian from the inherited DH transform chain; includes a finite-difference Jacobian check.
Milestone 03/kinematics/ur5e_velocity_kinematics.py	Implements forward velocity, inverse DLS velocity, and Jacobian singular-value diagnostics.
Milestone 03/kinematics/ur5e_acceleration_kinematics.py	Implements analytical $\dot{J}$, a finite-difference $\dot{J}$ check, forward acceleration, and inverse DLS acceleration.
Milestone 03/validation/validate_velocity_mujoco.py	Compares Python velocity kinematics with MuJoCo at five prescribed configurations and checks inverse reconstruction.
Milestone 03/validation/validate_acceleration_mujoco.py	Compares analytical $\dot{J}$, forward acceleration, pose finite-difference acceleration, and inverse acceleration with MuJoCo.
Milestone 03/visualization/ur5e_smooth_motion_demo.py	Runs a smooth point-to-point trajectory using the existing position actuators and records actual joint and end-effector quantities.
Milestone 03/visualization/plot_smooth_motion_results.py	Generates plots from the recorded smooth-motion CSV data.

6.2 Validation strategy

The static validation scripts load scene_surgical_m3.xml, identify the six robot joints and the attachment_site, assign prescribed joint states, and evaluate the model using MuJoCo. The Python implementation and MuJoCo results are compared at the same joint configuration. Five configurations are used: zero, home, configuration A, configuration B, and configuration C.

For velocity, the checks include the MuJoCo Jacobian-times-joint-rate result, a pose finite-difference velocity check, and inverse reconstruction. For acceleration, the checks include the analytical-versus-numerical $\dot{J}$ difference, the MuJoCo acceleration result, a pose finite-difference acceleration check, and inverse reconstruction.

The validation tolerances are those printed by the executed scripts. Velocity forward tolerance is 5×10^{-6} m/s or rad/s; inverse tolerance is 5×10^{-5} m/s or rad/s. Acceleration validation uses 5×10^{-5} per $\dot{J}$ entry, 5×10^{-6} m/s^2 or rad/s^2 for the forward MuJoCo comparison, and 5×10^{-5} m/s^2 or rad/s^2 for pose finite-difference and inverse checks.

6.3 Inherited position-kinematics verification

The position-level implementation was established in Milestone 02 and is included here as the base of the velocity and acceleration chain. The earlier report documents five static forward-kinematics tests with maximum position error 2.408×10^{-16} m and maximum orientation error 1.349×10^{-14} degrees. The Jacobian-transpose inverse-position solver was also tested on three targets with a 10^{-5} m tolerance [2].

Inverse-position target	Iterations	MuJoCo XYZ error (m)	Result
Test A	106	9.275×10^{-6}	PASS
Test B	252	9.715×10^{-6}	PASS
Home target	92	9.831×10^{-6}	PASS

The inverse-position solver constrains the attachment-site position. It does not guarantee that the end-effector orientation matches a separate orientation target.

7. MuJoCo Velocity and Acceleration Validation Results

7.1 Velocity validation

The five-configuration velocity validation completed successfully. The table reports the maximum error values printed by the executed validation script. The forward Jacobian-times-joint-rate and pose finite-difference checks are well below their configured tolerance, and the inverse reconstruction errors are below the inverse tolerance.

Configuration	MuJoCo $J\dot{q}$ error	Pose FD error	Inverse error	Result
Zero	6.939×10^{-17}	8.547×10^{-12}	2.927×10^{-11}	PASS
Home	1.665×10^{-16}	1.152×10^{-11}	4.235×10^{-11}	PASS
A	6.939×10^{-17}	2.418×10^{-11}	4.591×10^{-10}	PASS
B	1.110×10^{-16}	1.818×10^{-11}	1.281×10^{-10}	PASS
C	1.804×10^{-16}	1.736×10^{-11}	4.817×10^{-11}	PASS

All five MuJoCo velocity tests passed. The validation script uses a forward tolerance of 5×10^{-6} m/s or rad/s and an inverse tolerance of 5×10^{-5} m/s or rad/s.

7.2 Acceleration validation

The acceleration validation also passed for all five configurations. The analytical $\dot{J}$ was compared with a finite-difference approximation, and the forward and inverse acceleration calculations were compared with the corresponding MuJoCo and numerical checks.

Configuration	$\dot{J}$ error	MuJoCo accel. error	Pose FD error	Inverse error	Result
Zero	1.150×10^{-11}	1.822×10^{-12}	4.676×10^{-10}	5.000×10^{-11}	PASS
Home	6.790×10^{-11}	5.180×10^{-12}	5.488×10^{-10}	2.793×10^{-11}	PASS
A	1.703×10^{-11}	3.340×10^{-12}	1.231×10^{-9}	7.646×10^{-11}	PASS
B	2.289×10^{-11}	1.947×10^{-12}	8.446×10^{-10}	4.383×10^{-11}	PASS
C	1.099×10^{-11}	1.924×10^{-12}	1.842×10^{-9}	2.731×10^{-11}	PASS

All five MuJoCo acceleration tests passed. The configured tolerances were 5×10^{-5} per $\dot{J}$ entry, 5×10^{-6} m/s^2 or rad/s^2 for the forward MuJoCo acceleration comparison, and 5×10^{-5} m/s^2 or rad/s^2 for the pose finite-difference and inverse checks.

The very small errors are expected for a numerical comparison performed inside the same simulator model and coordinate convention. They demonstrate consistency between the Python kinematic equations and the MuJoCo kinematic quantities for the tested configurations; they should not be interpreted as physical-robot accuracy.

8. Smooth Point-to-Point Motion Demonstration

8.1 Trajectory definition

A separate dynamic demonstration was implemented to make the acceleration and deceleration phases visible in the simulation. The robot follows a coordinated six-joint trajectory from a home configuration to a nearby target. The scalar path parameter uses quintic time scaling:

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \quad \tau = t / T \tag{11}$$

$$qref(t) = qstart + (qgoal - qstart)s(\tau) \tag{12}$$

$$\dot{q}ref(t) = \Delta q\, \dot{s}(t), \quad \ddot{q}ref(t) = \Delta q\, \ddot{s}(t) \tag{13}$$

The first and second derivatives of the quintic scaling are zero at both ends of the movement. This gives a trajectory that starts from rest, increases its speed, then decreases its speed and reaches the target with zero reference velocity and acceleration.

Motion setting	Value used in supplied 2 ms dataset
Movement duration	6.00 s
Final hold	2.00 s
Total simulated duration	8.00 s
Physics timestep	0.002 s
CSV sample interval	0.002 s
Number of recorded samples	4,001
Initial joint configuration (rad)	[−1.5708, −1.5708, 1.5708, −1.5708, −1.5708, 0.0000]

Motion setting	Value used in supplied 2 ms dataset
Final joint target (rad)	[−1.8708, −1.4108, 1.3908, −1.4508, −1.6708, −0.2000]

8.2 Actuation method

The motion loop sends six coordinated position references to the existing MuJoCo position actuators. The actuator model remains a position servo with proportional-derivative behaviour. The script also computes an inverse-dynamics feedforward torque at the reference state and applies it as a position-command offset. Thus, the demo is position-reference control with dynamics feedforward; it is not a direct joint-velocity or joint-acceleration actuator command.

The gains printed by the script are Kp = [2000, 2000, 2000, 500, 500, 500] and Kd = [400, 400, 400, 100, 100, 100]. The feedforward term compensates the model's required generalized force at the reference state while retaining the existing actuator model. No position-control clipping or actuator-force saturation events were reported in the supplied run.

8.3 Measured motion profile

The supplied dataset shows a clear acceleration–deceleration profile. The average initial joint-speed norm was 0.00121 rad/s, approximately 0.8% of the measured peak. The peak joint-speed norm was 0.14403 rad/s at approximately 3.00 s. The final-hold average speed was effectively zero. The end-effector linear-speed peak was approximately 0.06005 m/s near the midpoint. The attachment-site position changed from approximately [−0.1340, 0.4920, 0.4880] m to [0.0438, 0.5613, 0.4814] m, a net displacement of approximately 0.191 m.

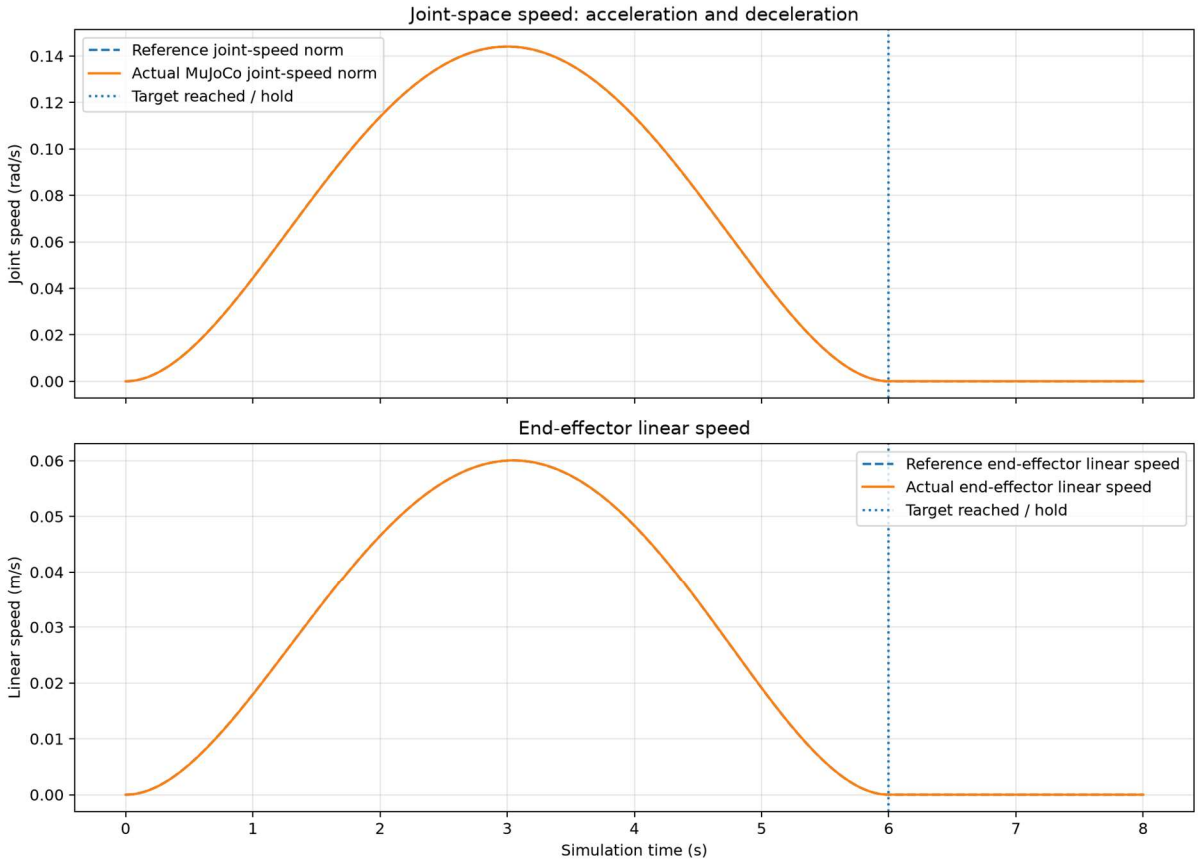


Figure 1. Joint-space speed norm and end-effector linear speed during the smooth point-to-point motion.

The reference and measured speed curves closely overlap. The speed increases from near zero, reaches a maximum around the middle of the six-second movement, and returns to zero at the target.

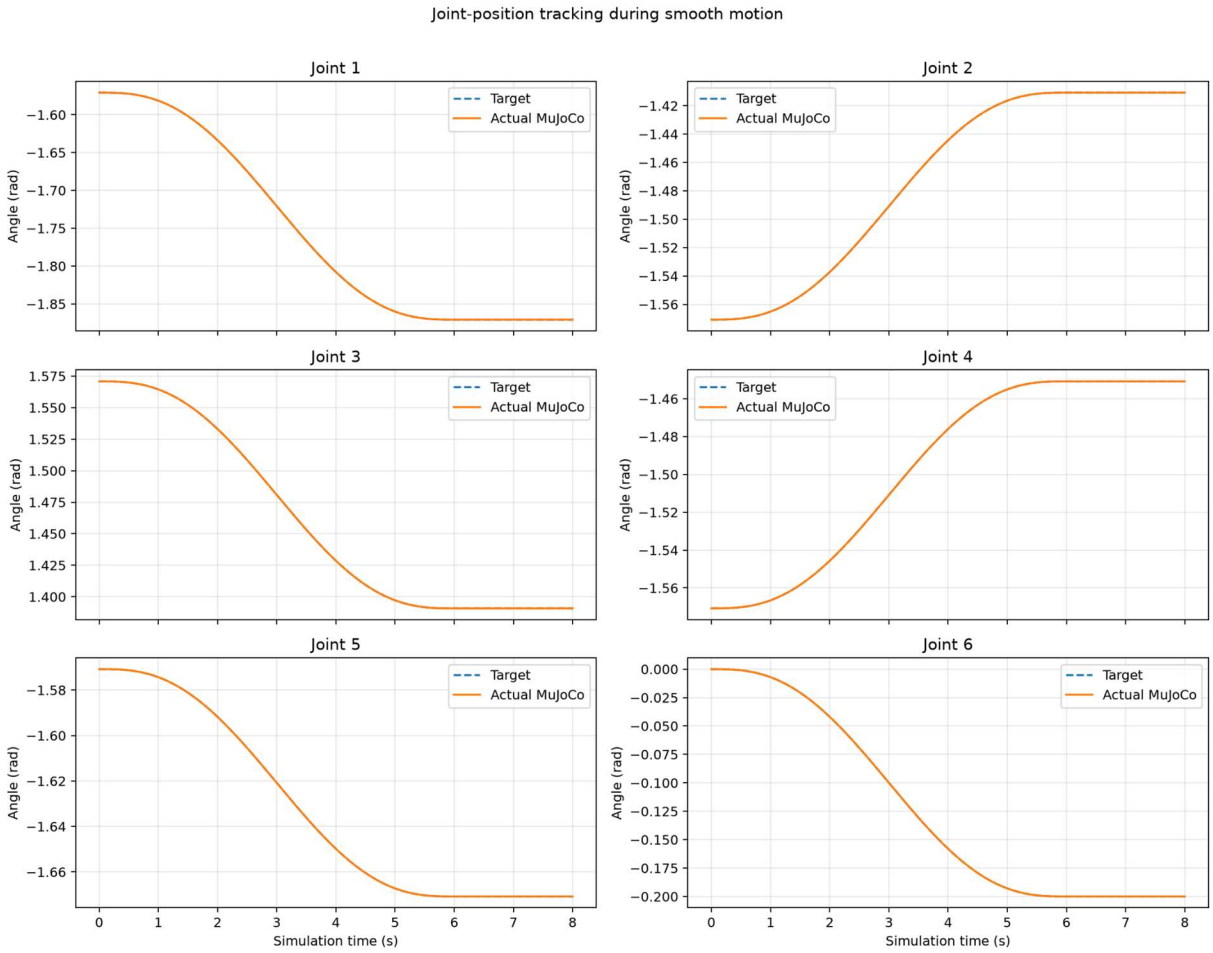


Figure 2. Target and actual MuJoCo joint positions for all six UR5e joints.

The actual joint-position traces closely follow the quintic references throughout the movement and final hold.

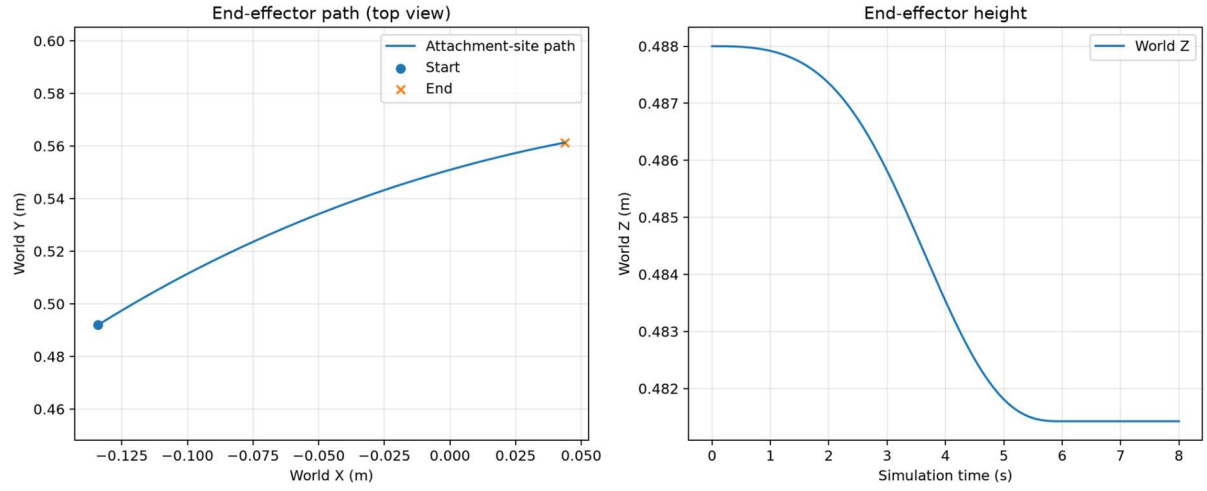


Figure 3. Attachment-site trajectory: top-view path and world-frame height.

The end-effector follows a smooth curved path between the initial and final attachment-site positions.

9. Dynamic Kinematic Agreement and Acceleration Measurement

9.1 Python–MuJoCo agreement at the same actual state

For every recorded sample, the script reads the actual joint position q, joint velocity $\dot{q}$, and MuJoCo generalized acceleration $\ddot{q}$. Python then calculates the attachment-site position, twist, and twist derivative. These values are compared with MuJoCo quantities evaluated at the same actual state. The maximum errors from the 4,001-sample dataset are:

Quantity	Maximum error	Configured tolerance	Result
Attachment-site position	6.947×10^{-16} m	1×10^{-5} m	PASS
Linear velocity	4.873×10^{-17} m/s	1×10^{-4} m/s	PASS
Angular velocity	1.015×10^{-16} rad/s	1×10^{-4} rad/s	PASS
Linear acceleration	3.717×10^{-13} m/s^2	1×10^{-3} m/s^2	PASS
Angular acceleration	9.161×10^{-13} rad/s^2	1×10^{-3} rad/s^2	PASS

These results demonstrate numerical agreement between the Python kinematics and the MuJoCo site/Jacobian calculations when both use the same actual joint state. They are not a measurement of how accurately the position actuators track the reference.

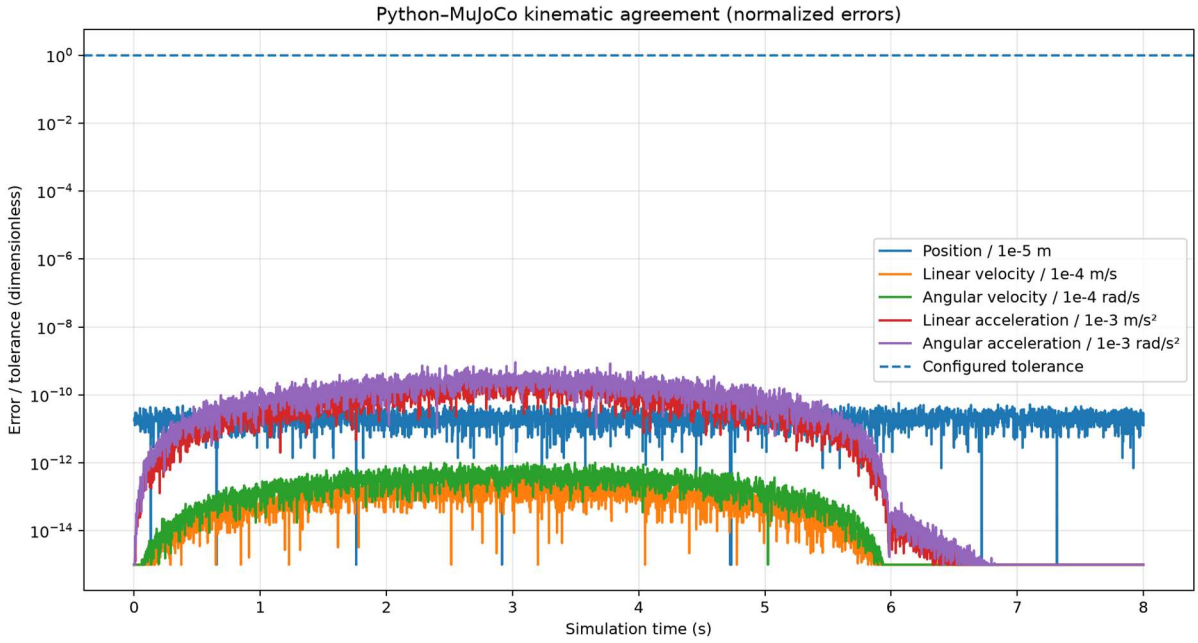


Figure 4. Python–MuJoCo kinematic error normalized by the configured tolerance.

All plotted normalized errors remain many orders of magnitude below the pass/fail threshold.

9.2 Position and velocity tracking of the smooth reference

Tracking quantity	RMS error	Maximum error
Joint position	9.024×10^{-6} rad	1.455×10^{-5} rad
Joint velocity	4.852×10^{-5} rad/s	7.904×10^{-5} rad/s

Tracking quantity	RMS error	Maximum error
End-effector linear velocity	2.024×10^{-5} m/s	3.328×10^{-5} m/s
End-effector angular velocity	1.810×10^{-5} rad/s	2.968×10^{-5} rad/s

The position and velocity tracking errors are small in the supplied run. The actuator-limit check reported zero position-control clipping events and zero actuator-force saturation events across 24,006 actuator checks.

9.3 Two acceleration measurements and their interpretation

The smooth-motion experiment retains two acceleration definitions rather than treating them as identical. The first is the instantaneous generalized acceleration qacc returned by MuJoCo at the recorded state after forward dynamics evaluation. The second is a numerical derivative of the sampled actual joint-velocity trace $\dot{q}$, calculated using finite differences. These quantities can differ because they represent different numerical measurements around a discrete physics integration step; the supplied test does not establish a single causal explanation for the discrepancy.

For the supplied 2 ms dataset, the finite-difference derivative of actual $\dot{q}$ follows the reference joint acceleration closely (RMS error 6.054×10^{-5} rad/s^2; maximum 2.442×10^{-4} rad/s^2). The finite-difference-derived end-effector linear and angular accelerations also follow their reference profiles closely. In contrast, the instantaneous qacc and the finite-difference derivative of $\dot{q}$ differ substantially (RMS norm error approximately 6.844×10^{-2} rad/s^2; maximum approximately 1.115×10^{-1} rad/s^2).

Acceleration diagnostic	RMS error	Maximum error
Finite-difference $\dot{q}$ vs reference $\ddot{q}$	6.054×10^{-5} rad/s^2	2.442×10^{-4} rad/s^2
Finite-difference end-effector linear acceleration vs reference	2.537×10^{-5} m/s^2	9.795×10^{-5} m/s^2
Finite-difference end-effector angular acceleration vs reference	2.278×10^{-5} rad/s^2	8.994×10^{-5} rad/s^2
Instantaneous MuJoCo qacc vs finite-difference $\dot{q}$	6.844×10^{-2} rad/s^2	1.115×10^{-1} rad/s^2

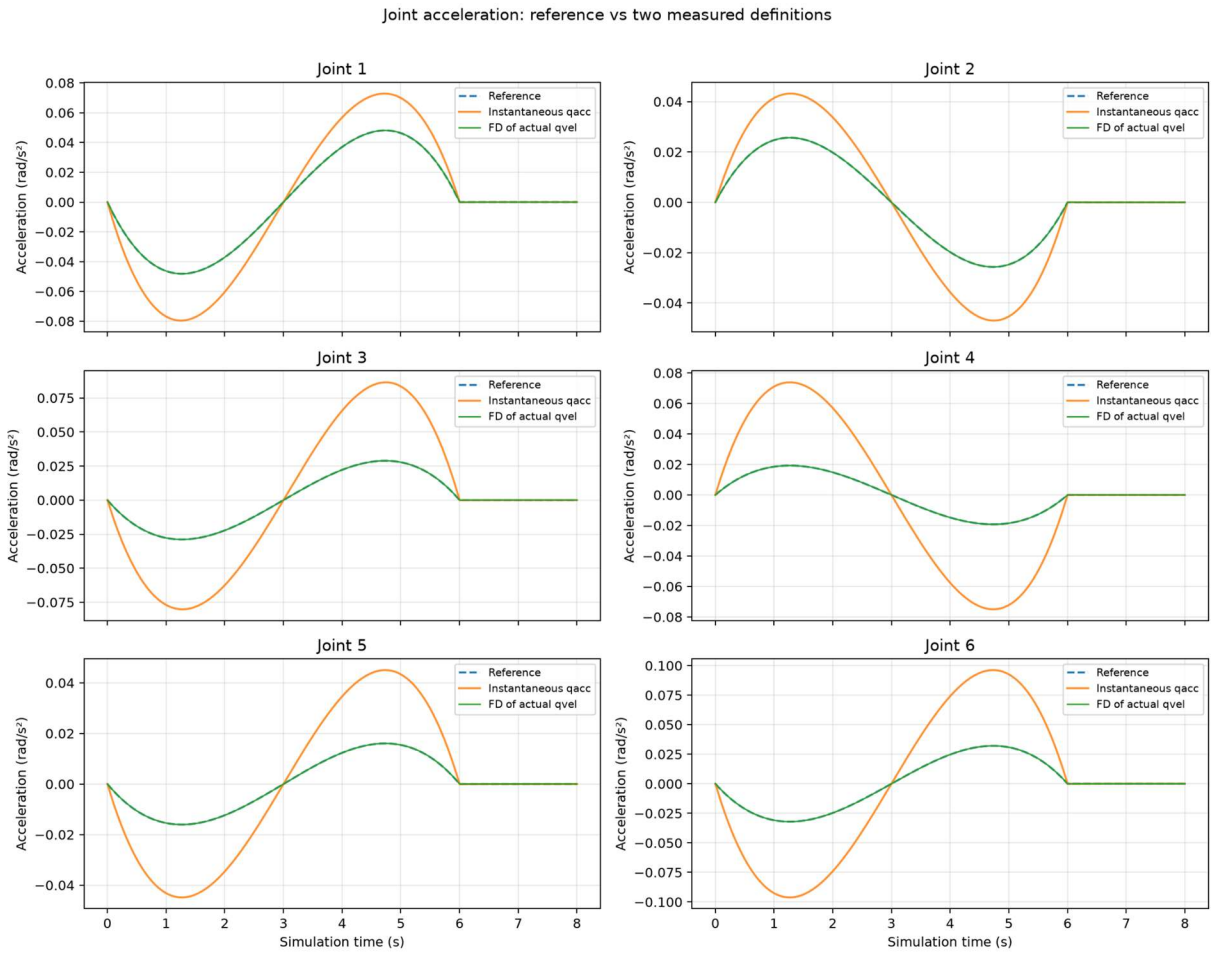


Figure 5. Joint acceleration comparison: reference, instantaneous MuJoCo qacc, and finite-difference derivative of actual $\dot{q}$.

The finite-difference acceleration trace follows the reference closely. The instantaneous qacc trace is retained separately because it differs from the finite-difference estimate.

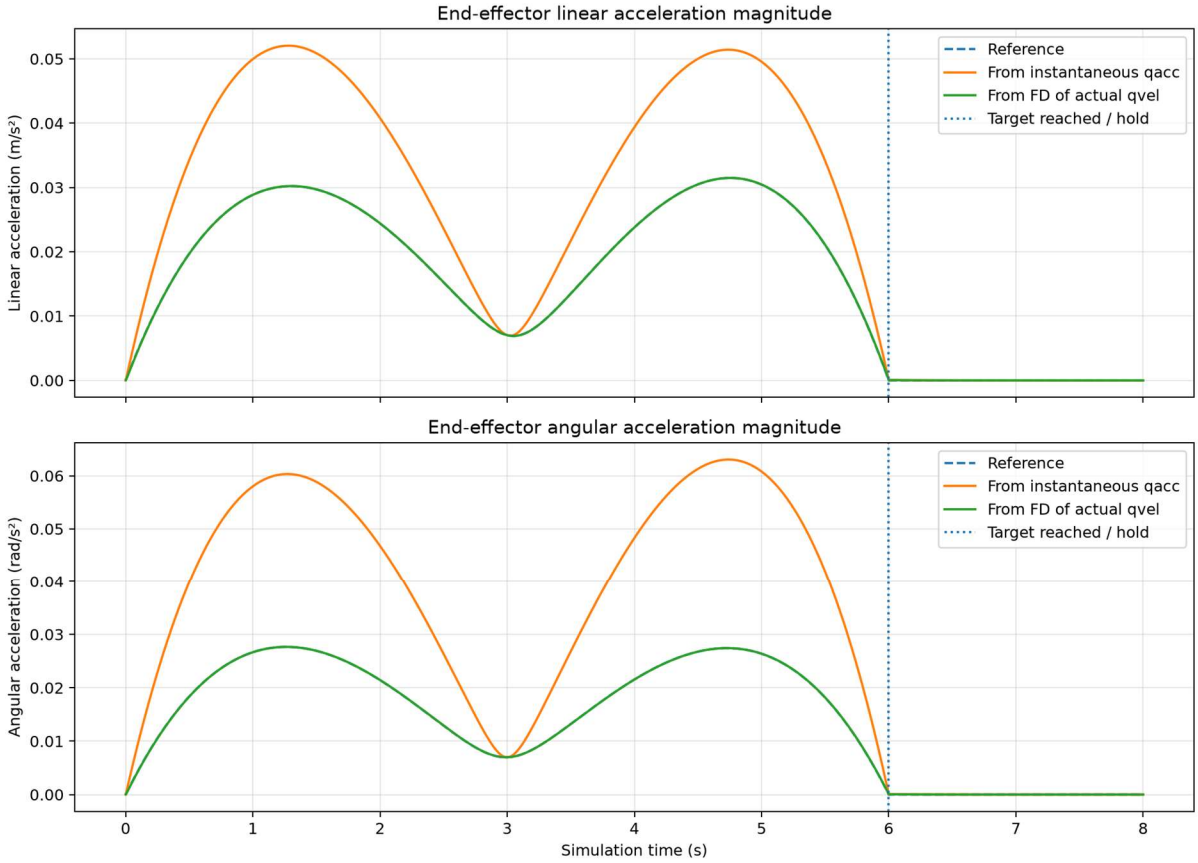


Figure 6. End-effector linear and angular acceleration magnitudes from the reference, instantaneous qacc, and finite-difference actual $\dot{q}$.

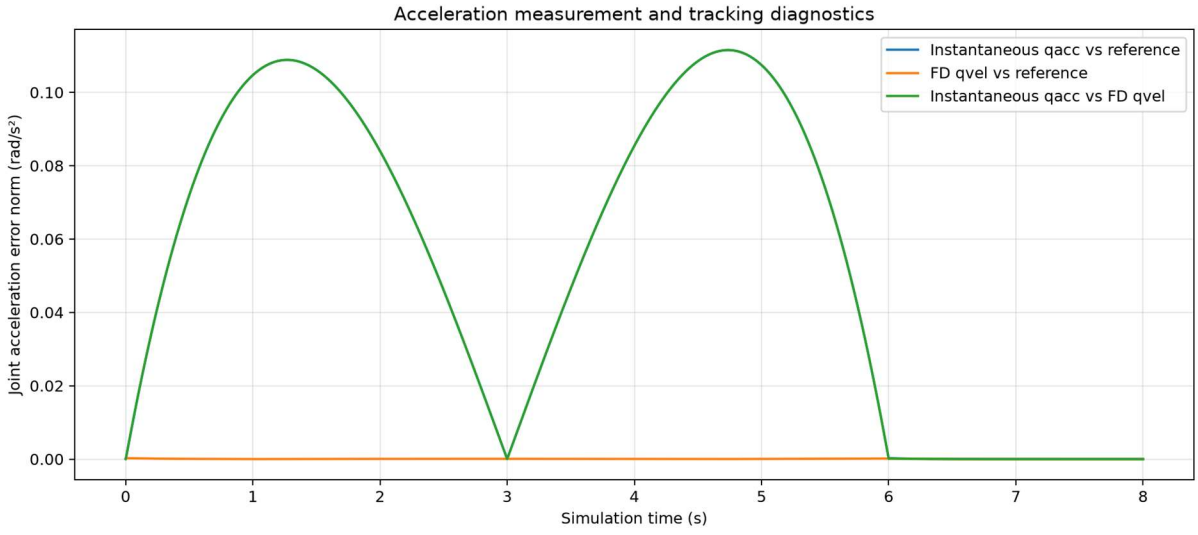


Figure 7. Acceleration measurement and tracking diagnostics across the complete movement.

The orange finite-difference-versus-reference error remains small, while the qacc-versus-finite-difference diagnostic is visibly larger. This is why the report separates kinematic-equation agreement from acceleration tracking.

The main kinematic pass/fail check remains the comparison of the Python and MuJoCo acceleration equations evaluated at the same q, $\dot{q}$, and $\ddot{q}$. That comparison passed with maximum errors below 10^{-12} in the supplied run. The finite-difference plots provide a separate trajectory-tracking diagnostic. The instantaneous qacc discrepancy should be acknowledged in the discussion and, if further refinement is performed, investigated using the

simulator's integration and sampling conventions before describing instantaneous qacc as the measured derivative of $\dot{q}$.

10. Surgical Scene, GUI, and Reproducibility

10.1 Surgical simulation scene

The smooth-motion demonstration uses the updated MuJoCo scene file Milestone 01/Robot Model/scene_surgical_m3.xml and evaluates the attachment_site of the UR5e. The broader project application is a simulated surgical-tool handover workstation. Milestone 03 also requires continued GUI/environment development and separate videos of the created GUI [1]. The numerical plots in this report document the kinematics and smooth-motion experiment; they do not replace the required GUI demonstration video.

10.2 Software and execution context

Item	Recorded configuration
Simulator	MuJoCo 3.2.0
Programming language	Python (project virtual environment)
Numerical library	NumPy
Execution context	Ubuntu environment under WSL2, as used in the project terminal
Scene	Milestone 01/Robot Model/scene_surgical_m3.xml
End-effector reference	attachment_site

10.3 Reproduction commands

Run the following commands from the repository root after activating the project's Python virtual environment. The paths below match the project structure used for the supplied results.

Forward/inverse velocity Python demonstration

```
python "Milestone 03/kinematics/ur5e_velocity_kinematics.py"
```

Velocity validation against MuJoCo

```
python "Milestone 03/validation/validate_velocity_mujoco.py" --scene "Milestone 01/Robot
Model/scene_surgical_m3.xml"
```

Forward/inverse acceleration Python demonstration

```
python "Milestone 03/kinematics/ur5e_acceleration_kinematics.py"
```

Acceleration validation against MuJoCo

```
python "Milestone 03/validation/validate_acceleration_mujoco.py" --scene "Milestone 01/Robot
Model/scene_surgical_m3.xml"
```

Smooth-motion simulation (GUI, 6 s movement + 2 s hold)

```
python "Milestone 03/visualization/ur5e_smooth_motion_demo.py" --scene "Milestone 01/Robot
Model/scene_surgical_m3.xml" --move-duration 6 --hold-duration 2 --sample-interval 0.02 --camera overview -
-csv "Milestone 03/Results/Smooth_Motion/video_demo.csv"
```

Generate plots from a recorded CSV

```
python "Milestone 03/visualization/plot_smooth_motion_results.py" --csv "Milestone
03/Results/Smooth_Motion/video_demo.csv" --outdir "Milestone 03/Figures/Smooth_Motion/Video_Demo"
```

The supplied 4,001-sample numerical dataset was collected with a 2 ms sample interval and headless execution. The GUI command above is intended for visual demonstration and uses a 20 ms sample interval by default in this

command. If a longer 12 s movement is used for the final video, rerun the experiment, regenerate the plots, and update the movement-duration values and figures in this report so that the report and video describe the same run.

11. Discussion and Limitations

- The Python geometric Jacobian and analytical Jacobian time derivative agree with the MuJoCo calculations for the five prescribed configurations within the configured tolerances.
- The inverse velocity and inverse acceleration calculations use damped least squares. Near singular configurations or for unattainable Cartesian requests, the solution may be approximate; the reconstruction residual should always be inspected.
- The smooth-motion script uses position references sent to existing position actuators, with inverse-dynamics feedforward applied as a position-command offset. It is not direct velocity control or direct acceleration control.
- The instantaneous MuJoCo qacc and finite-difference derivative of actual $\dot{q}$ differ in the supplied trajectory experiment. The report therefore presents both definitions and does not claim that the instantaneous qacc trace matches the reference acceleration.
- The validation is simulation-based and does not establish performance on a physical UR5e robot.
- The inverse-position test constrains end-effector position only; it does not guarantee a specified end-effector orientation.
- The project brief also requires GUI/environment videos. The kinematic plots in this report do not replace those separate visual deliverables.

These limitations define the scope of the evidence. The reported PASS results apply to the implemented equations and tested simulator configurations; they should not be generalized to untested joint states, hardware operation, or collision-constrained autonomous handover.

12. Conclusion

Milestone 03 extends the team's UR5e position-kinematics foundation with a complete Python implementation of forward and inverse velocity and acceleration kinematics. The geometric Jacobian is constructed from the selected standard-DH frame assignment, and its analytical time derivative is used to account for configuration-dependent changes during motion. The inverse problems are solved with an SVD-based damped least-squares pseudoinverse.

The five-configuration MuJoCo validation completed successfully for both velocity and acceleration. The inherited position-level tests also provide continuity with Milestone 02. In the smooth-motion experiment, the robot follows a quintic point-to-point reference, accelerates during the first half of the movement, decelerates during the second half, and remains at the final pose during the hold interval. The measured speed profile and joint-position traces support the intended visible motion behaviour.

The dynamic results also show why validation metrics must be interpreted carefully: Python and MuJoCo kinematics agree at the same actual state, while instantaneous qacc and finite-difference acceleration are distinct measurements in the sampled simulation. Retaining this distinction makes the report more accurate and provides a clear direction for any additional acceleration-measurement refinement.

References

[1] German University in Cairo, Faculty of Engineering & Materials Science, Mechatronics Department, “MCTR911 – Robotics Programming, Winter 2026: Project Description,” Milestone 03 requirements and deliverables, p. 8.

[2] Project team, “MCTR911 – Milestone 02: UR5e Robot Kinematics, ROS 2–MuJoCo Integration, and Motion Visualization,” team report, 13 October 2026.

[3] MuJoCo, version 3.2.0, official documentation. https://mujoco.readthedocs.io/en/3.2.0/ (accessed during project development).

Submission Completion Checklist

Before exporting or submitting the report, the team should complete the following:

- Replace all blank team-member fields with the correct names and student IDs.
- Enter the correct tutorial group and supervisor/evaluator details if required.
- Update the table of contents in Microsoft Word.
- Confirm that the figures correspond to the final CSV and the motion duration used in the submitted video.
- If the extended 12 s motion is used in the final video, regenerate the figures and update the corresponding numerical values before submission.
- Place the report, Python source files, validation scripts, and required narrated/GUI videos in the Milestone 03 GitHub folder.